

Entwicklerdokumentation der Entertainment-Erweiterung für IFE

Stand: 05.05.2026, durch Gervithrall Systems

Auftraggeber: Novaris Cabin Systems GmbH
Friedrich-List-Platz 1
01069 Dresden

Ansprechpartner*in: Lea Wagner

E-Mail: lwagner@novaris-cabinystems.de

Telefon: 0351 4620

Auftragnehmer: Gervithrall Systems GmbH
Perlickstraße 1
04103 Leipzig

Ansprechpartner*in: Lucas Rumann

E-Mail: lucasr@gervithrall-systems.de

Telefon: 0351 6482642

Inhaltsverzeichnis

1. Einführung
2. Versionen und Tools / Plugins
 - 2.1 Grundlegende Entwicklungsumgebung
 - 2.2 Verwendete Plugins und Abhängigkeiten
 - 2.3 Externe Tools
3. Projektstruktur und Inhalt
 - 3.1 Ordnerbaum und wichtigste Ordner
 - 3.2 Klassendiagramm
 - 3.3 JavaDoc
4. Wichtigste Befehle für Build und Deployment
5. Code Conventions
6. Tests
7. Buildscript
8. Informationen für weitere Entwicklung
 - 8.1 Programmtrennung
 - 8.2 Vorgehensweise für die Erstellung eines weiteren Spiels
 - 8.3 Hinzufügen von neuen Sprachen oder CI-Themes

1. Einführung

Diese Entwicklerdokumentation dient als zentrale technische Referenz für das Java-Projekt der Entertainment-Erweiterung „4-Connect“ im Inflight-Entertainment-System (IFE). Sie richtet sich an Entwickler, die das Projekt verstehen, erweitern oder warten möchten.

In dieser Dokumentation werden insbesondere die verwendeten Versionen, die Entwicklungsumgebung sowie die grundlegende Projektstruktur beschrieben. Ziel ist es, einen schnellen Einstieg in das Projekt zu ermöglichen und ein einheitliches Verständnis der Systemarchitektur und Arbeitsweise zu schaffen.

Detaillierte Informationen zur Ordnerstruktur sowie zu den konkreten Inhalten des Repositories können ergänzend in der README-Datei eingesehen werden: README

2. Versionen und Tools / Plugins

Die Bewertungen und Begründungen für die Auswahl der 11 wichtigsten Werkzeuge können in folgendem Dokument gefunden werden:

Entscheidung zur Entwicklungsumgebung und genutzten Tools für die Erstellung der Entertainment-Erweiterung für IFE

(auch zu finden unter: `/final/toolvergleich.md`)

2.1 Grundlegende Entwicklungsumgebung

- Java 21 LTS (21.0.x)

Die Anwendung basiert auf Java 21 (Long Term Support). Diese Version bietet langfristige Stabilität und wird auch in der Zielumgebung (IFE) verwendet.

- JavaFX (21.0.2)

JavaFX wird zur Erstellung der grafischen Benutzeroberfläche eingesetzt. Es ermöglicht die Umsetzung der UI inklusive Layout, Interaktion und Darstellung.

- Maven (3.9.15)

Maven dient als Build-Tool zur Verwaltung von Abhängigkeiten, zum Kompilieren des Projekts sowie zur Erstellung des finalen Artefakts (JAR-Datei).

- JUnit 5 (5.11.0)

JUnit wird für Unit-Tests verwendet. Es ermöglicht die automatisierte Überprüfung der Spiellogik und trägt zur Sicherstellung der Codequalität bei.

2.2 Verwendete Plugins und Abhängigkeiten

- Maven: 3.9.15

Build-Management-Tool zur Strukturierung und Automatisierung des Projektes.

- JavaFX: 21.0.2

Framework zur Umsetzung der Benutzeroberfläche.

- JUnit: 5.11.0

Framework zur Durchführung von Unit-Tests

- Surefire: 3.2.5

Maven-Plugin zur automatischen Ausführung der Tests während des Build-Prozesses.

- JAR Plugin: 3.3.0

Erstellt die ausführbare JAR-Datei des Projekts.

- Resource Plugin: 3.3.1

Verarbeitet und kopiert Ressourcen wie FXML, CSS und Sprachdateien in das Build-Artefakt.

- JaCoCo: 0.8.11

Tool zur Messung der Testabdeckung (Code Coverage).

- Shade: 3.5.1

Erzeugt eine sogenannte „Fat-JAR“, welche alle benötigten Abhängigkeiten enthält.

- ProGuard: 2.6.1

Wird zur Obfuscation des Codes eingesetzt, um die Anwendung vor Reverse Engineering zu schützen.

2.3 Externe Tools

- Git / GitHub

Das Projekt wird über Git versioniert und auf GitHub verwaltet. Git ermöglicht Versionskontrolle, Branching sowie Zusammenarbeit im Team.

Link zum Repo: <https://github.com/realLucas01/SEGR11>

- Adobe XD Version XD 58

Wird für die Erstellung von UI-Wireframes und Design-Prototypen verwendet.

- PlantUML Version 2.18.1

Tool zur Erstellung von UML-Diagrammen in textbasierter Form, die versionierbar im Repository abgelegt werden.

- IDE

Die Wahl der Entwicklungsumgebung ist freigestellt. Im Projekt wurden verwendet:

- Eclipse IDE Version 2025-09
 - IntelliJ IDEA Version 2025.2.6.x

3. Projektstruktur und Inhalt

3.1 Ordnerbaum und wichtigste Ordner

```
java_projekt/IFE_Entertainment/
├── src/
│   ├── main/
│   │   ├── java/SEG11/IFE_Entertainment/
│   │   │   ├── FourConnect/           - Spiellogik für Vier Gewinnt
│   │   │   ├── GameCore/             - Allgemeine Spiel-Interfaces
│   │   │   ├── Infrastructure/        - Sprachen und Theme-Verwaltung
│   │   │   ├── UIController/         - Allgemeine UI Controller und Interfaces
│   │   │   └── UIFourConnectController/ - Controller spezifisch für Vier Gewinnt
│   │   └── resources/
│   │       ├── i18n/                 - Sprachdateien
│   │       └── SEG11/IFE_Entertainment/ - FXML-Dateien und CSS
│   └── test/                         - Unit Tests
├── target/                          - Speicherort für Build-Artefakte (.jar, Reports)
├── pom.xml                          - Maven Build-Konfiguration
└── proguard.config                  - Konfiguration für Obfuscation
```

Die Struktur folgt dem klassischen Maven-Standard und trennt klar zwischen Quellcode, Ressourcen und Tests. Besonders wichtig ist die Trennung von Spiellogik (Backend), UI und Infrastrukturkomponenten.

3.2 Klassendiagramm

Es dient zur Übersicht der wichtigsten Klassen, ihrer Beziehungen sowie der Systemarchitektur.



Klassendiagramm

Das vollständige Klassendiagramm des Systems ist in dieser Datei zu finden: Klassendiagramm

(auch zu finden unter: `/final/Klassendiagramm.svg`)

Und die PlantUML-Projektdatei für das Klassendiagramm hier: FourConnect_ClassDiagram

(auch zu finden unter: `/uml_projekt/FourConnect_ClassDiagram.puml`)

3.3 JavaDoc

Die generierte JavaDoc-Dokumentation kann unter folgendem Link eingesehen werden: JavaDoc

(oder auch: `/final/JavaDoc/index.html`)

Sie enthält detaillierte Beschreibungen aller Klassen, Methoden und Schnittstellen.

4. Wichtigste Befehle für Build und Deployment

Start des Programms aus der Konsole:

```
java -jar IFE_Entertainment-[Versionsnummer]-obfuscated.jar
```

Das Programm ist mit einem automatischen Buildscript ausgestattet, das automatisch alte Builds löscht und die .jar, so wie alle dazugehörigen Dokumente (JavaDoc und Test-Coverage Report) generiert. Die genauere Funktionsweise dieses Scriptes ist in diesem Dokument, unter Punkt 7 - Buildscript, erklärt. Es wird mit folgendem Befehl eingesetzt:

```
mvn clean site install
```

5. Code Conventions

Im Projekt wird der Google Java Code Style verwendet, siehe:

<https://google.github.io/styleguide/javaguide.html>

* (oder auch: `/final/Google Java Style Guide.pdf`) *

Zusätzlich gelten folgende Regeln:

- Funktions-, Klassen- und Variablennamen sind in Englisch zu schreiben
- JavaDoc und Kommentare sind in Deutsch zu verfassen

Ausnahme: rechtliche Disclaimer im Kopf von Klassen werden in Englisch verfasst
Ziel ist eine konsistente, gut lesbare und wartbare Codebasis.

6. Tests

Tests dienen insbesondere zur Absicherung der Spiellogik und zur Vermeidung von Regressionen. Für alle zentralen Klassen werden Unit-Tests erstellt, die Testabdeckung wird mit JaCoCo überwacht.

Anforderungen:

- Mindest-Coverage: 50%
- Bei Unterschreitung kann der Build fehlschlagen (je nach Konfiguration)

Testausführung in der Konsole:

```
mvn test
```

7. Buildscript

Das Projekt verwendet ein automatisiertes Buildsystem auf Basis von Maven und befindet sich daher in der pom.xml innerhalb der `<build>` `</build>` Artefakte. Die wichtigsten Informationen zur Funktionsweise der einzelnen Plugins/Executions sind ebenfalls in der pom.xml kommentiert.

Es wird über folgenden Befehl in der Konsole ausgeführt:

```
mvn clean site install
```

und generiert dabei hauptsächlich 3 Artefakte:

- die Ausführbare .jar mitsamt allen Dependencies, unter `/final/IFE_Entertainment-[Versionsnummer]-obfuscated.jar`
- das Javadoc, unter `/final/JavaDoc`
- Den Bericht für die Testabdeckung von JaCoCo, unter `/final/Coverage`

Hinweis: Diese Artefakte befinden sich nach Ausführen des Buildscripts ebenfalls in `\java_projekt\IFE_Entertainment\target`. Zusätzlich sind in diesem Ordner dann auch die `IFE_Entertainment-[Versionsnummer].jar` (ohne JavaFX Bibliotheken) und die `IFE_Entertainment-[Versionsnummer]-executable.jar` (nicht obfuscated) zu finden.

Die grobe Funktionsweise des Buildscripts und seiner Plugins ist wie folgt:

- **Kompilierung des Codes** durch das maven-compiler-plugin
- **Testen:**
 - maven-surefire-plugin → führt JUnit-Tests aus
 - jacoco-maven-plugin → misst Testabdeckung und erstellt dazugehörigen Report
- **Packaging:**
 - maven-jar-plugin → Standard-JAR mit Main-Class
 - maven-shade-plugin → Fat-JAR, also inkl. aller Dependencies aufgrund dessen das JavaFX kein Teil der SDK mehr ist
 - proguard-maven-plugin → obfuscated JAR
- **Automatisches Kopieren** der Ausgabedateien (obfuscated JAR, Javadoc und Test-Report) in das Zielverzeichnis `/final` durch das maven-resources-plugin.

* (das Javadoc befindet sich unter `/final/JavaDoc` und der Test-Coverage-Report unter `/final/Coverage`. Es muss dann jeweils die index.html in einem Browser geöffnet werden) *

8. Informationen für weitere Entwicklung

8.1 Programmtrennung

Die Anwendung folgt einer klaren Trennung der Verantwortlichkeiten: FXML & CSS → ausschließlich für die Darstellung der Benutzeroberfläche Controller → Verarbeitung von Benutzereingaben und Steuerung der UI Backend bzw. GameCore → Enthält die komplette Spiellogik und ist unabhängig von der UI

Diese Trennung ermöglicht eine bessere Wartbarkeit und erleichtert die Erweiterung des Systems.

8.2 Vorgehensweise für die Erstellung eines weiteren Spiels

1. Erstellen von 2 neuen Packages mit folgenden benennungen:
 - {Spielname}Controller, für die eigene Implementierung der Controller
 - {Spielname}, für die eigene Implementierung des Backends

Der Inhalt der geschweiften Klammern ist durch den Namen des eigenen Spiels zu ersetzen.

2. Implementieren aller Interfaces aus den Modularen Packages:
 - Für die Controller sind die Interfaces aus UIController zu implementieren.
 - Für die Klassen des Backends sind die Interfaces aus GameCore zu implementieren.

***Ausnahme:** Das Interface IMoveStrategy aus GameCore, dieses muss nur implementiert werden, wenn es einen automatisierten bzw. Bot-Spieler gibt.*

3. Hinzufügen und Implementieren von weiteren Klassen die Spezifisch nur für das eigene Spiel gebraucht werden.
4. Erstellen der benötigten .fxml für das UI
5. Erweitern des Hauptmenüs um die Kachel des eigenen Spiels

8.3 Hinzufügen von neuen Sprachen oder CI-Themes

Genauere Informationen und Anleitungen zum hinzufügen neuer Themes für die CI-Anpassung oder dem hinzufügen neuer Sprachen sind in der Admindoku in Sektion 6 - Konfiguration zu finden.

(auch zu finden unter: </final/admindoku.pdf>)